The Hegemoney organization has been flagged.
Because of that, your organization is hidden from the public. If you believe this is a mistake, contact support to have your organization's status reviewed.

Branch: master QCSuper / README.md Find file Copy path

p1-mmr Merge branch 'master' of https://github.com/P1sec/QCSuper 4a9fa93 3 days ago

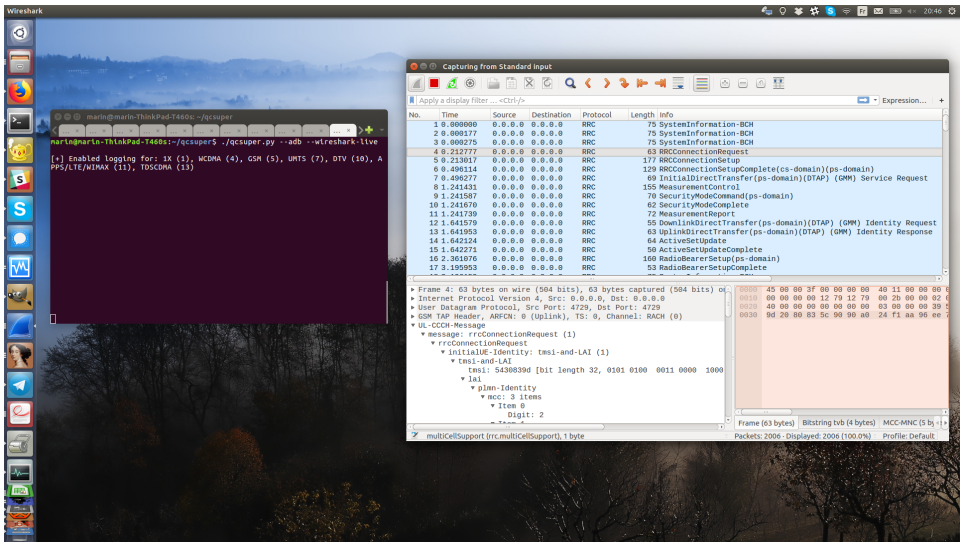
3 contributors

246 lines (171 sloc) 15.3 KB Raw Blame History

QCSuper

QCSuper is a tool communicating with Qualcomm-based phones and modems, allowing to **capture raw 2G/3G/4G** radio frames, among other things.

It will allow you to **generate PCAP** captures of it using either a rooted Android phone, an USB dongle or an existing capture in another format.



After having installed it, you can plug your rooted phone in USB and using it is as simple as:

```
./qcsuper.py --adb --wireshark-live
```

It uses the Qualcomm Diag protocol, also called QCDM or DM (Diagnostic Monitor) in order to communicate with your phone's baseband.

You want support, to report that you device works or does not work or you'd like to join development doing research on the Diag protocol? You can come talk on IRC (`#qcsuper` at Freenode) or open a Github issue.

Table of contents

- **Installation**
  - Ubuntu and Debian installation
  - Windows installation
- Supported protocols
- **Usage notice**

Annexes:

- Using QCSuper with an USB modem
- Supported devices
- Related tools using the Diag protocol

Blog post/demo: Presenting QCSuper: a tool for capturing your 2G/3G/4G air traffic on Qualcomm-based phones

More documentation:

- The Diag protocol
- QCSuper architecture

Installation

QCSuper was tested and developed on Ubuntu 16.04, 18.04 and Windows 7. It depends on a few Python modules.

To use it, your phone must be rooted or expose a diag service port over USB. In order to check for compatibility with your phone, look up the phone's model on a site like GSMArena and check whether it has a Qualcomm processor.

In order to open PCAP files produced by QCSuper, you can use any Wireshark 2.x for 2G/3G frames, but you need at least Wireshark 2.5.x for 4G frames (and 2.6.x for individual NAS messages decrypted out of 4G frames). Ubuntu currently provides a recent enough build for all versions.

Ubuntu and Debian installation

Open a terminal and type the following:

```
# Download QCSuper
git clone git@github.com:P1sec/QCSuper.git qcsuper
cd qcsuper

# Install dependencies
sudo apt install python3-pip wireshark
```

```
sudo pip3 install --upgrade pyserial crcmod https://gith
```

Windows installation

On Windows, you will need to download and install your phone's USB drivers from your phone model. There is no generic way, search for your phone's model + "USB driver" or "ADB driver" on Google for instructions.

Then, you need to ensure that you can reach your device using `adb`. You can find a tutorial on how to download and setup `adb` here. The `adb shell` command must display a prompt to continue.

Then, follow these links on order to:

- Install Python 3.6 or more recent (be sure to check options to include it into PATH, install it for all users and install pip)
- Install Wireshark 2.6 or more recent
- Download and extract QCSuper

To install the required Python modules, open your command prompt and type:

```
pip3 install --upgrade pyserial crcmod https://github.co
```

Still in your command prompt, move to the directory containing QCSuper using the `cd` command. You can then execute commands (which should start with `qcsuper.py` instead of `./qcsuper.py`).

Supported protocols

QCSuper supports capturing a handful of mobile radio protocols. These protocols are put after a GSMTAP header, a standard header (encapsulated into UDP/IP) permitting to identify the protocol, and GSMTAP packets are put into a PCAP file that is fully analyzable using Wireshark.

2G/3G/4G protocols can be broken into a few "layers": layer 1 is about the digital radio modulation and multiplexing, layer 2 handles stuff like fragmentation and acknowledgement, layer 3 is the proper signalling or user data.

QCSuper allows you most often to capture on layer 3, as it is the most pratical to analyze using Wireshark, and is what the Diag protocol provides natively (and some interesting information is here).

- 2G (GSM): Layer 3 and upwards (RR/...)
- 2.5G (GPRS and EDGE): Layer 2 and upwards (MAC-RLC/...) for data acknowledgements
- 3G (UMTS): Layer 3 and upwards (RRC/...)
  - Additionally, it supports reassembling SIBs (System Information Blocks, the data broadcast to all users) in separate GSMTAP frames, as Wireshark currently can't do it itself: flag `--reassemble-sibs`
- 4G (LTE): Layer 3 and upwards (RRC/...)
  - Additionally, it supports putting decrypted NAS message, which are embedded encrypted embedded into RRC packet, in additional frames: flag `--decrypt-nas`

By default, the IP traffic sent by your device is not included, you see only the signalling frames. You can include the IP traffic you generate using the `--include-ip-traffic` option (IP being barely the layer 3 for your data traffic in 2G/3G/4G, at the detail that its headers may be compressed (ROHC) and a tiny PPP header may be included).

The data traffic you send uses a channel different from the signalling traffic, this channed is setup through the signalling traffic; QCSuper should thus show you all details relevant to how this channel is initiated.

Usage notice

In order to use QCSuper, you specify one input (e.g: `--adb` (Android phone), `--usb-modem`) and one or more modules (`--wireshark-live` for opening Wireshark, `--pcap-dump` for writing traffic to a PCAP file, `--info` for generic information about the device...).

A few commands you can type are:

```
# Open Wireshark directly, using a rooted Android phone a
./qcsuper.py --adb --wireshark-live

# Same, but dump to a PCAP file instead of opening Wiresh
./qcsuper.py --adb --pcap-dump /tmp/my_pcap.pcap

# Same, but using an USB modem exposing a Diag serial por
sudo ./qcsuper.py --usb-modem /dev/ttyHS2 --wireshark-liv
```

Here is the current usage notice for QCSuper:

```
usage: qcsuper.py [-h] [--cli] [-v]
                  (--adb | --usb-modem TTY_DEV |
--dlf-read DLF_FILE | --json-geo-read JSON_FILE)
                  [--info] [--pcap-dump PCAP_FILE]
[--wireshark-live]
                  [--memory-dump OUTPUT_DIR] [--dlf-
dump DLF_FILE]
                  [--json-geo-dump JSON_FILE]
[--decoded-sibs-dump]
                  [--reassemble-sibs] [--decrypt-nas]
[--include-ip-traffic]
                  [--start MEMORY_START] [--stop
```

```
MEMORY_STOP]

A tool for communicating with the Qualcomm DIAG
protocol (also called QCDM or
DM).

optional arguments:
  -h, --help            show this help message and
exit
  --cli                 Use a command prompt, allowing
for interactive
                        completion of commands.
  -v, --verbose         Add output for each received
or sent Diag packet.

Input mode:
  Choose an one least input mode for DIAG data.

  --adb                 Use a rooted Android phone
with USB debugging enabled
                        as input (requires adb).
  --usb-modem TTY_DEV   Use an USB modem exposing a
DIAG pseudo-serial port
                        through USB.
  --dlf-read DLF_FILE   Read a DLF file generated by
QCSuper or QXDM, enabling
                        interoperability with vendor
software.
  --json-geo-read JSON_FILE
                        Read a JSON file generated
using --json-geo-dump.

Modules:
  Modules writing to a file will append when it
already exists, and consider
  it Gzipped if their name contains ".gz".

  --info                Read generic information about
the baseband device.
  --pcap-dump PCAP_FILE
                        Generate a PCAP file
containing GSMTAP frames for
                        2G/3G/4G, to be loaded using
```

```
Wireshark.
  --wireshark-live      Same as --pcap-dump, but
directly spawn a Wireshark
                        instance.
  --memory-dump OUTPUT_DIR
                        Dump the memory of the device
(may not or partially
                        work with recent devices).
  --dlf-dump DLF_FILE   Generate a DLF file to be
loaded using QCSuper or
                        QXDM, with network protocols
logging.
  --json-geo-dump JSON_FILE
                        Generate a JSON file
containing both raw log frames
                        and GPS coordinates, for
further reprocessing. To be
                        used in combination with
--adb.
  --decoded-sibs-dump   Print decoded SIBs to stdout
(experimental, requires
                        pycrate).

PCAP generation options:
  To be used along with --pcap-dump or --wireshark-
live.

  --reassemble-sibs     Include reassembled UMTS SIBs
as supplementary frames,
                        also embedded fragmented in
RRC frames.
  --decrypt-nas         Include unencrypted LTE NAS as
supplementary frames,
                        also embedded ciphered in RRC
frames.
  --include-ip-traffic  Include unframed IP traffic
from the UE.

Memory dumping options:
  To be used along with --memory-dump.

  --start MEMORY_START  Offset at which to start to
dump memory (hex number),
```

```
                          by default 00000000.
  --stop MEMORY_STOP    Offset at which to stop to
dump memory (hex number),
                          by default ffffffff.
```

Specifying `-` to pipe data from stdin or towards stdout is supported (gzipped content may not be detected).

Using QCSuper with an USB modem

You can use QCSuper with an USB modem exposing a Diag port using the `--usb-modem <device>` option, where `<device>` is the name of the pseudo-serial device on Linux (such as `/dev/ttyUSB0`, `/dev/ttyHS2` and other possibilites) or of the COM port on Windows (such as `COM3`).

Please note that in most setups, you will need to run QCSuper as root in order to be able to use this mode, notably for handling serial port interference.

If you don't know which devices under `/dev` expose the Diag port, you may have to try multiple of these. You can try to auto-detect it by stopping the ModemManager daemon (`sudo systemctl stop ModemManager`), and using the following command: `sudo ModemManager --debug 2>&1 | grep -i 'port is QCDM-capable'` then Ctrl-C.

Please note that if you're not able to use your device with for example ModemManager in the first place, it is likely that it is not totally setup and that it will not work neither with QCSuper. A few possible gotchas are:

- You didn't apply the proper mode switching command for your device.
-

If you bought a device that previously had a SIM from a different operator, your device may be sim-locked. You may have to use the unlock code from the former operator and submit it to the device, as if it was a PIN code: `sudo mmcli -i 0 --pin=<your_unlock_code>`

If your Qualcomm-based USB device doesn't expose a Diag port by default, you may need to type the following through the AT port in order to enable the Diag port:

```
AT$QCDMG
```

Please note that only one client may communicate with the Diag port at the same time. This applies to two QCSuper instances, or QCSuper and ModemManager instances.

If ModemManager is active on your system, QCSuper will attempt to dynamically add an udev rule to prevent it to access the Diag port and restart its daemon, as it's currently the best way to achieve this. It will suppress this rule when closed.

Supported devices

QCSuper was successfully tested with:

- Sony Xperia Z (Phone) - 4G - works out of the box after rooting an enabling adb
- Nexus 6P (Phone) - 4G - works out of the box after rooting an enabling adb
- ZTE MF823 (USB Modem) - 4G - may require to mode-switch the device to CDC-WDM, set the device to factory mode, then execute the AT command mentioned above
- ZTE MF667 (USB Modem) - 3G, 2011 - should work out of the box (may require mode switching)
- Option Icon 225 (USB Modem) - 3G, 2008
- Novatel Ovation MC998D (USB Modem)

- ZTE WCDMA Technologies MSM MF110/MF627/MF636 (USB Modem)
- ZTE 403zt (USB Modem) - 4G
- OnePlus One and 3 (Phones)
- Andromax A16C3H (Phone)
- Samsung Galaxy S4 GT-I9505 (Phone)

Is it however aiming to be compatible with the widest possible range of devices based on a Qualcomm chipset, for the capture part.

Do no hesitate to report whether your device is successfully working or not through the IRC channel, or opening a Github issue.

Related tools using the Diag protocol

There are a few other open tools implementing bits of the Diag protocol, serving various purposes:

- ModemManager: the principal daemon enabling to use USB modems on Linux, implements bits of the Diag protocol (labelled as QCDM) in order to retrieve basic information about USB modem devices.
- SnoopSnitch (specifically gsm-parser): chiefly an Android application whose purpose is to detect potential attacks on the radio layer (IMSI catcher, fake BTS...). It also have a secondary feature to capture some signalling traffic to PCAP, which does not provide exactly the same thing as QCSuper (LTE traffic isn't encapsulated in GSMTAP for example, device support may be different).
    - diag-parser: A Linux tool that derivates from the PCAP generation feature from SnoopSnitch, somewhat improved, designed to work with USB modems.
- MobileInsight: this Android application intends to parse all kinds of logs output by Qualcomm and Mediatek devices

(not only those containing signalling information, but also proprietary debugging structures), and dumping these to a specific XML representation format. Does not provide user-facing PCAPs (but formerly used Wireshark as a backend for converting certain protocol information to XML).

- qcombbdbg: A debugger for the Qualcomm baseband setting up itself by hooking a Diag command, through using the Diag command that allows to write to memory, for the Option Icon 225 USB modem.
- OpenPST: A set of tools related to Qualcomm devices, including a GUI utility allowing, for example, to read data on the tiny embedded filesystem accessible through Diag (EFS).
- SCAT: A tool with similar GSMTAP generation abilities, taking as input a serial port, also supporting Samsung Exynos.